

QASP: Quantum-Armored Service Protocol for Secure Autonomous Agent Communication and Resource Authorization

Chintakunta Shreya, Woo Jon Hou Ainsley, Wonna Tun, Wong Pui Ling, Xue Yufeng
Singapore University of Technology and Design (SUTD)
50.012 Networks
February 2026

Abstract—The proliferation of autonomous AI agents acting on behalf of humans creates a critical need for a secure, standardized networking protocol governing agent-to-agent (A2A) and agent-to-server (A2S) communication. Current agent protocols (Google’s A2A, Anthropic’s MCP, ANP, AGP) focus on task orchestration and tool invocation but lack three essential properties: (1) cryptographically verifiable agent identity tied to human ownership, (2) fine-grained, delegatable resource authorization via capability tokens, and (3) resistance to quantum computing threats.

We propose QASP (Quantum-Armored Service Protocol), an application-layer protocol that provides PKI-based agent identity, scoped capability tokens for resource authorization, delegation chains with attenuation, and metered resource accounting—all secured with post-quantum cryptography using CRYSTALS-Dilithium (ML-DSA) for digital signatures and CRYSTALS-Kyber (ML-KEM) for key encapsulation, with hybrid classical/PQ modes for backward compatibility. We introduce a Universal Agent Resource Model (ARM) that provides a canonical vocabulary for agent capabilities regardless of wire-protocol adoption, cryptographically composable delegation chains with formal token algebra, a tamper-evident accountability layer with dispute resolution, economic incentive alignment via micropayment channels, decentralized identity via a `did:qasp` DID method, service discovery through QASP-Discover, explicit interoperability bridges to A2A and MCP, and a concrete standardization roadmap targeting RFC-style specification. This paper describes the protocol architecture, handshake design, message formats, trust model, threat model, formal security arguments, and a concrete implementation plan demonstrating two agents negotiating compute access with signed post-quantum capability tokens.

Index Terms—Post-quantum cryptography, agent protocols, capability-based authorization, ML-DSA, ML-KEM, delegation chains, autonomous agents, crypto agility, decentralized identity, micropayments

I. INTRODUCTION

By 2027, an estimated 50% of knowledge workers will use AI agents for daily tasks [1]. These agents will need to authenticate to services and to each other, request resources such as compute cycles, API calls, storage, and data access, delegate authority (e.g., “my travel agent can book flights up to \$500 on my behalf”), and account for usage through metered billing, audit trails, and spending limits.

No existing protocol addresses all of these requirements simultaneously. Current approaches treat agents as either dumb

API clients (MCP [3]) or task-passing peers (A2A [2]) without a coherent identity, authorization, and accounting layer. Meanwhile, the cryptographic foundations of every deployed protocol rely on RSA/ECDSA/ECDH—algorithms that a cryptographically relevant quantum computer (CRQC) would break, compromising all agent identities and authorization tokens.

NIST estimates a 50% probability of a CRQC capable of breaking RSA-2048 by 2035 [6]. Agent identity certificates and capability tokens issued today using classical cryptography are vulnerable to *harvest-now, decrypt-later* (HNDL) attacks. An adversary recording agent protocol traffic today could retroactively forge identities, fabricate capability tokens, and rewrite authorization histories once quantum computers mature. For a protocol designed to underpin economic transactions between agents, this is unacceptable.

We propose QASP, a protocol that fills this gap by combining PKI-based agent identity, capability-based authorization, delegation chains, metered accounting, economic settlement, decentralized identity, service discovery, and post-quantum cryptography into a unified application-layer protocol. Beyond the wire protocol itself, QASP introduces a *Universal Agent Resource Model* (ARM) that serves as a canonical vocabulary for expressing agent capabilities, resource scopes, and constraints—providing value as a *ground-truth ontology* even to systems that do not adopt the QASP wire format. Crucially, QASP is designed as a *security and authorization layer* that complements rather than replaces existing agent orchestration protocols such as A2A and MCP.

II. RELATED WORK

A. Google A2A (Agent-to-Agent Protocol)

Google’s A2A protocol [2] enables agents built on different frameworks to communicate and collaborate on tasks. It provides Agent Cards for capability discovery, task lifecycle management, and streaming support. However, identity is based on OAuth 2.0/OpenID Connect with no agent-specific PKI or owner-binding certificates, no native resource authorization or capability token system, and no post-quantum cryptography considerations.

TABLE I
FEATURE COMPARISON OF AGENT PROTOCOLS

Feature	A2A	MCP	ANP	AGP	QASP
Agent-specific PKI	×	×	~	×	✓
Owner-binding certs	×	×	×	×	✓
Capability tokens	×	×	×	×	✓
Delegation chains	×	×	×	×	✓
Resource metering	×	×	×	×	✓
Post-quantum crypto	×	×	×	×	✓
Peer-to-peer	✓	×	✓	×	✓
Hybrid PQ/classical	×	×	×	×	✓
Crypto agility	×	×	×	×	✓
Universal ARM	×	×	×	×	✓
DID identity	×	×	✓	×	✓
Service discovery	✓	×	~	×	✓
Economic settlement	×	×	×	×	✓

B. Anthropic MCP (Model Context Protocol)

MCP [3] standardizes how AI models connect to tools, data sources, and external systems. While it offers clean tool/resource abstraction and widespread adoption, it is client-server only with no peer-to-peer agent communication, a trivial identity model, no delegation or capability tokens, and security deferred entirely to transport (HTTPS/TLS).

C. ANP (Agent Network Protocol)

ANP [4] provides open agent networking with DID-based identity and meta-protocol negotiation. Its DID methods still rely on classical cryptography, resource authorization is underspecified, and there are no capability token standards, delegation chains, or post-quantum considerations.

D. AGP (Agent Gateway Protocol)

AGP [5] is an enterprise gateway for agent communication. It is gateway-centric (single point of failure/trust), uses coarse-grained ACL-based authorization, has no post-quantum support, and is tightly coupled to enterprise infrastructure.

E. Summary

Table I summarizes the feature gap that QASP addresses.

III. PROTOCOL DESIGN

A. Architecture Overview

QASP operates at the application layer (Layer 7) on top of a reliable transport (TCP or QUIC). It defines six sub-protocols:

- **QASP-ID:** Identity and PKI management (including `did:qasp`)
- **QASP-Discover:** Service discovery and capability advertisement
- **QASP-Shake:** Handshake and secure channel establishment
- **QASP-Cap:** Capability tokens and authorization
- **QASP-Meter:** Metering and accounting
- **QASP-Settle:** Economic incentive alignment and payment settlement

[Architecture Diagram: QASP protocol stack showing QASP-ID, QASP-Discover, QASP-Shake, QASP-Cap, QASP-Meter, and QASP-Settle sub-protocols over CBOR framing and TCP/QUIC transport]

Fig. 1. QASP protocol architecture and layer stack.

All messages use CBOR-encoded binary framing for compactness. The protocol entities include *Agents* (autonomous software entities with ML-DSA keypairs), *Owners* (humans or organizations holding root keypairs), *Servers* (resource providers), and optionally a *Certificate Authority* for enterprise deployments.

B. Universal Agent Resource Model (ARM)

A key contribution of QASP is the *Agent Resource Model* (ARM), a canonical ontology for expressing what an agent *can do* with a resource. ARM is intentionally decoupled from the QASP wire format so that it can serve as a *ground-truth vocabulary* for any agent ecosystem—even those that do not adopt QASP’s handshake or framing. If two systems agree on ARM semantics, capability tokens and authorization policies become interoperable regardless of the underlying transport.

1) *Resource Identifiers:* Every resource is named by a structured URI:

qasp://{provider}/{resource}/{subresource} (1)

Examples: `qasp://acme-cloud/gpu/a100`, `qasp://data-co/dataset/medical-records/row-level`. The hierarchical structure enables prefix-based attenuation: a token scoped to `qasp://acme-cloud/gpu/*` covers all GPU sub-resources.

2) *Action Verbs:* ARM defines seven canonical verbs that span the lifecycle of agent–resource interaction:

- `read / write` — data-plane access
- `exec` — invoke a computation or tool
- `delegate` — pass a subset of rights to another agent
- `charge` — bill against a spending budget
- `attenuate` — narrow an existing token’s scope
- `revoke` — invalidate a token and its descendants

Every capability token contains a subset of these verbs; the server enforces that the requested action is in the token’s verb set.

3) *First-Class Constraints:* ARM elevates six constraint dimensions to first-class citizens of every token:

Because constraints are structurally typed and machine-comparable, the token algebra (Section III-J) can compute intersections and detect escalation attempts purely by comparing constraint fields—no policy-engine side-channel is needed.

TABLE II
ARM CONSTRAINT DIMENSIONS

Constraint	Semantics
time	Validity window (not-before / not-after)
quantity	Max units consumable (e.g., tokens, vCPU-h)
rate	Max units per time window
spend	Monetary ceiling in base currency
data-scope	Row/column/partition restrictions
purpose	Free-text or enum binding (audit, research, ...)

4) *ARM as Ground Truth*: Even if an ecosystem uses OAuth scopes, XACML, or ad-hoc JSON policies internally, mapping those policies *to and from* ARM yields a portable, auditable representation. We envision ARM registries (akin to IANA media-type registries) where providers publish their resource hierarchies and supported verbs, enabling automated capability discovery across heterogeneous agent platforms.

C. Decentralized Identity with *did:qasp*

Traditional PKI binds agent identity to a centralized Certificate Authority, creating a single point of failure and trust. QASP introduces the *did:qasp* DID method [18], enabling self-sovereign agent identity with *optional* CA anchoring for enterprise environments.

1) *DID Method Specification*: A *did:qasp* identifier is derived deterministically from the agent's ML-DSA-65 public key:

```
did:qasp:<base58btc(SHA-384 (ML-DSA-65-PK) [0:32])>
```

The corresponding DID Document follows the W3C DID Core specification [18]:

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/suites/pq-2025"
  ],
  "id": "did:qasp:z6MkhaXgBZD...",
  "verificationMethod": [{
    "id": "did:qasp:z6MkhaXgBZD...#key-1",
    "type": "MLDSAVerificationKey2025",
    "controller": "did:qasp:z6MkhaXgBZD...",
    "publicKeyMultibase": "z4BWwf..."
  }],
  "authentication": [
    "did:qasp:z6MkhaXgBZD...#key-1"
  ],
  "capabilityDelegation": [
    "did:qasp:z6MkhaXgBZD...#key-1"
  ],
  "service": [{
    "id": "did:qasp:z6MkhaXgBZD...#qasp",
    "type": "QASPEndpoint",
    "serviceEndpoint": "qasp://example.com:4433"
  }],
  "ownerBinding": {
    "ownerDID": "did:qasp:z6MkownerXYZ...",
    "delegationProof": "<ML-DSA-65 signature>"
  }
}
```

2) *Resolution and Bootstrapping*: DID Documents are resolved via three mechanisms in priority order:

- 1) **Direct exchange**: During QASP-Shake, peers include their DID Documents in the handshake, enabling fully decentralized, zero-infrastructure identity.

- 2) **QASP-Discover**: DID Documents are embedded in PQ-signed capability advertisements broadcast via DNS-SD/mDNS (Section III-D).
- 3) **DID resolver network**: For asynchronous resolution, DID Documents are published to a distributed hash table (DHT) or a Verifiable Data Registry (VDR). The registry entry is signed by the agent's ML-DSA-65 key, ensuring only the key holder can update it.

3) *Owner Binding*: The *ownerBinding* field links the agent DID to its owner's DID via a cross-signed proof: the owner signs $\langle \text{agent_did}, \text{owner_did}, \text{permissions}, \text{expiry} \rangle$ with their ML-DSA-65 key. This replaces the centralized CA chain with a verifiable delegation from owner to agent while preserving the same security guarantees.

4) *Optional CA Anchoring*: Enterprise deployments MAY anchor *did:qasp* identities to a traditional CA by including a *caAttestation* field containing an ML-DSA-65-signed certificate from the organization's CA. This provides a migration path: organizations start with CA-anchored DIDs and optionally transition to fully self-sovereign operation.

5) *Key Rotation*: DID Documents include a *keyRotation* section specifying the next public key hash, enabling pre-committed key rotation. The rotation is effected by publishing an updated DID Document signed by *both* the current and next keys, preventing hostile takeover even if the current key is compromised (the attacker would also need the pre-committed next key).

D. QASP-Discover: Service Discovery Sub-Protocol

Before agents can handshake, they must *find* each other. QASP-Discover fills this bootstrapping gap with a PQ-secured service discovery mechanism.

1) *Discovery Mechanisms*: QASP-Discover operates in three modes:

- 1) **Local (mDNS/DNS-SD)**: Agents on the same network segment broadcast DNS-SD service records [19] with service type *_qasp._tcp*. The TXT record contains a PQ-signed capability advertisement (see below). This enables zero-configuration discovery in LAN/edge environments.
- 2) **Wide-area (DNS)**: Agents publish SRV and TXT records under their domain (e.g., *_qasp._tcp.agent.example.com*). DNSSEC provides transport integrity; the TXT payload provides PQ-authenticated content integrity.
- 3) **Registry**: A QASP registry service (akin to a service mesh control plane) maintains a directory of agent DIDs, capabilities, and endpoints. The registry is queried via QASP-Shake-protected channels, and all entries are PQ-signed by the advertising agent.

2) *PQ-Signed Capability Advertisement*: Each advertisement is a CBOR-encoded, ML-DSA-65-signed structure:

```
CapabilityAd = {
  version:      uint,
  agent_did:    tstr,           ; did:qasp identifier
  endpoints:    [+ Endpoint],   ; transport addresses
```

[Certificate/DID Hierarchy Diagram: Root CA or Owner DID → Owner Certificate/DID → Agent Certificate/DID → Ephemeral Session Key]

Fig. 2. QASP identity hierarchy (PKI and DID modes).

```
capabilities: [+ CapEntry], ; ARM resource/verb
  pairs
constraints: ConstraintSet, ; global limits
did_document: bytes, ; embedded DID Document
timestamp: uint, ; Unix epoch
ttl: uint, ; seconds
signature: bytes ; ML-DSA-65 over all
  above
}
CapEntry = {
  resource: tstr, ; ARM URI
  verbs: [+ tstr] ; ARM verb set
}
```

The signature prevents advertisement spoofing: a quantum adversary cannot forge a valid advertisement without the agent's ML-DSA-65 private key. The `ttl` field enables cache expiry, and the `timestamp` prevents replay of stale advertisements.

3) *Discovery-to-Handshake Pipeline*: Upon receiving a valid capability advertisement, a client agent:

- 1) Verifies the ML-DSA-65 signature over the advertisement.
- 2) Extracts the DID Document and resolves the owner binding.
- 3) Checks that the advertised capabilities match its needs.
- 4) Initiates QASP-Shake to the advertised endpoint, including the advertisement hash in the ClientHello for binding.

E. Trust Model and PKI

QASP supports two complementary identity models: centralized PKI (Section III) and decentralized `did:qasp` (Section III-C). In the PKI model, an optional Root CA issues Owner Certificates (ML-DSA-65 signed), which in turn sign Agent Certificates. In the DID model, owner-to-agent binding is expressed via cross-signed DID proofs. Both models produce equivalent *trust chains* that the QASP-Cap subsystem verifies uniformly.

Ephemeral session keys are established via ML-KEM-768 per session. Each agent identity (whether certificate or DID-based) contains the agent ID (UUID v7 or DID), issuer reference (owner public key fingerprint or owner DID), ML-DSA-65 public key (1952 bytes), validity period, owner-binding proof, permitted capability classes, maximum delegation depth, and spending limits.

F. Post-Quantum Handshake (QASP-Shake)

The QASP-Shake handshake establishes a mutually authenticated, quantum-resistant secure channel through a three-message exchange:

TABLE III
QASP-SHAKE STATE MACHINE

State	Event	Next State	Action
Idle	connect()	HelloSent	Send ClientHello
Idle	recv(CH)	HelloRecv	Validate CH
HelloSent	recv(SH)	Authing	Verify server sig
HelloSent	timeout	Error	Send Alert(timeout)
HelloSent	recv(Alert)	Error	Log & close
HelloRecv	validate_ok	SHSent	Send ServerHello
HelloRecv	validate_fail	Error	Send Alert(bad_ch)
SHSent	recv(CA)	Verifying	Verify client sig
SHSent	timeout	Error	Send Alert(timeout)
Authing	verify_ok	Established	Derive keys
Authing	verify_fail	Error	Send Alert(auth_fail)
Verifying	verify_ok	Established	Derive keys
Verifying	verify_fail	Error	Send Alert(auth_fail)
Established	close()	Idle	Send close_notify
Error	—	Idle	Reset state

- 1) **ClientHello**: Protocol version, ML-KEM-768 encapsulation key, supported cipher suites, client nonce, optionally an X25519 public key for hybrid mode, and optionally a QASP-Discover advertisement hash for discovery binding.
- 2) **ServerHello**: Selected cipher suite, ML-KEM-768 ciphertext (yielding shared secret), server nonce, server identity (certificate chain or DID Document), optionally X25519 public key, and ML-DSA-65 signature over the transcript.
- 3) **ClientAuth**: Client identity (certificate chain or DID Document), ML-DSA-65 signature over the transcript, and optionally a capability token request.

Key derivation uses HKDF-SHA-384:

$$SS = \text{HKDF}(\text{ML-KEM}_{ss} \| X25519_{ss}, N_c \| N_s, \text{"QASP-v1"}) \quad (2)$$

where N_c and N_s are client and server nonces. The resulting symmetric key protects the channel using AES-256-GCM or ChaCha20-Poly1305. In PQ-only mode, the X25519 component is omitted.

1) *QASP-Shake State Machine*: The handshake is specified as a deterministic finite state machine with explicit timeout, retransmission, and error handling:

Timeout and retransmission. The default handshake timeout is 30 seconds. Over unreliable transports (UDP/QUIC), ClientHello and ServerHello are retransmitted with exponential backoff (1s, 2s, 4s, max 3 retries). A `message_seq` counter in each handshake message enables deduplication.

Version mismatch. If the server does not support the client's proposed version, it responds with `Alert(version_mismatch, supported_versions=[...])`, allowing the client to retry with a compatible version. The maximum negotiation rounds is 2; failure after that terminates with `Alert(fatal_version_mismatch)`.

[Handshake Sequence Diagram: Agent A ↔ Server B showing ClientHello, ServerHello, ClientAuth, and Finished messages with ML-KEM-768 key exchange and ML-DSA-65 mutual authentication]

Fig. 3. QASP-Shake handshake protocol.

G. Capability Token Structure

Capability tokens are the core authorization primitive—signed, scoped, time-bound, and attenuable. Each CBOR-encoded token contains: version, token ID (UUID v7), issuance and expiry timestamps, issuer/subject/audience public key fingerprints (or DIDs), resource URI (ARM format), ARM verb set, ARM constraints (Table II), parent token reference, delegation chain, purpose binding, allowed toolchain classes, authority chains (for multi-owner scenarios, Section III-H), and an ML-DSA-65 signature.

For example, a token might authorize: “Agent Alpha may exec up to 2 vCPU-hours on `qasp://acme-cloud/gpu/a100`, spending at most \$2.50, valid for 2 hours, `purpose=training`, with delegation permitted up to depth 2.”

1) Advanced Capability Token Features:

a) *Token Aggregation Algebra*: Agents may present *multiple* capability tokens simultaneously to compose permissions. The aggregation follows a *union-with-constraint-intersection* algebra:

$$\text{Agg}(T_1, T_2) = \begin{cases} V = V_1 \cup V_2 \\ R = R_1 \cup R_2 \\ C = C_1 \sqcap C_2 \text{ (on shared dimensions)} \end{cases} \quad (3)$$

where V is the verb set, R is the resource set, and $C_1 \sqcap C_2$ denotes the component-wise minimum constraint on shared dimensions (e.g., time validity is the intersection of both windows; spending limit is the minimum). Unshared dimensions are inherited from the token that defines them. The server verifies each token independently and then computes the aggregation; no single token need authorize the full request.

b) *Threshold Delegation (M-of-N Authorization)*: For high-value capabilities, QASP supports M -of- N threshold authorization. An owner group $(O_1, \dots, O_N)$ collectively issues a token requiring M signatures for activation. We employ a *PQ threshold signature* construction over ML-DSA-65 using Shamir secret sharing over the signing key [25]: each owner holds a key share sk_i , and M owners produce partial signatures that are combined into a single valid ML-DSA-65 signature. The resulting token is indistinguishable from a single-signer token to verifiers, but requires distributed authorization to create.

c) *Temporal Capability Evolution*: Tokens may include a `temporal_attenuation` schedule that automatically reduces permissions over time:

```
temporal_attenuation: {
  schedule: [
    { at: "T+1h", spend_limit: 0.75 },
    { at: "T+2h", spend_limit: 0.50 },
    { at: "T+3h", spend_limit: 0.25 }
  ],
  policy: "linear_decay" ; or "step", "exponential"
}
```

The server evaluates the schedule at each request, applying the most restrictive constraint whose time threshold has been reached. This enables “use it or lose it” patterns where tokens naturally attenuate as they approach expiry.

d) *Cross-Domain Delegation*: When Agent Alpha (owned by Alice) needs to delegate to Agent Beta (owned by Bob), a *trust boundary crossing* occurs. QASP handles this via a *cross-domain delegation token* that carries:

- 1) Alice’s original token T_A authorizing Alpha.
- 2) Alpha’s attenuation $T_{A \rightarrow B} = \text{att}(T_A, \Delta)$ scoped for Beta.
- 3) A *cross-domain endorsement*: Alice signs $\langle T_{A \rightarrow B}, \text{Bob’s DID} \rangle$, explicitly authorizing a foreign-owner agent to exercise a subset of capabilities.
- 4) Bob’s countersignature accepting accountability for Beta’s actions under this token.

The server verifies the full chain: Alice $\rightarrow$ Alpha $\rightarrow$ Beta, with both owner signatures covering the cross-domain boundary. This prevents unauthorized cross-domain delegation while enabling legitimate multi-party workflows.

H. Multi-Owner and Organizational Delegation

Enterprise environments require capabilities governed by multiple authorities simultaneously. QASP supports *multi-owner tokens* where multiple authority chains must ALL validate.

1) *Hierarchical Policy Composition*: A multi-owner token carries an `authority_chains` array, each entry being an independent delegation chain:

```
authority_chains: [
  { role: "employee", chain: [T_employee] },
  { role: "department", chain: [T_dept_policy] },
  { role: "corporate", chain: [T_corp_spending] }
]
composition: "all" ; all chains must validate
```

The server verifies each chain independently and computes the *intersection* of all constraints. A request is authorized only if it falls within the intersection of all authority chains.

2) *Example: Corporate Travel Agent*: Consider a travel-agent authorized by:

- 1) **Employee** (Alice): “Book flights for me, up to \$2000/trip.”
- 2) **Department policy**: “Marketing employees may book economy class only, max 3 trips/month.”
- 3) **Corporate spending cap**: “Total Q1 travel budget: \$50,000 remaining: \$12,340.”

TABLE IV
QASP MESSAGE TYPES

ID	Message	Direction
0x01	ResourceRequest	$A \rightarrow S$
0x02	ResourceGrant	$S \rightarrow A$
0x03	ResourceDeny	$S \rightarrow A$
0x04	ResourceRelease	$A \rightarrow S$
0x05	TokenRevocation	$O/S \rightarrow *$
0x06	RevocationNotice	$* \rightarrow *$
0x07	DelegationRequest	$A_1 \rightarrow A_2$
0x08	DelegationGrant	$A_2 \rightarrow A_1$
0x09	MeterReport	$S \rightarrow A$
0x0A	MeterAck	$A \rightarrow S$
0x0B	CapabilityQuery	$A \rightarrow S$
0x0C	CapabilityAdvertise	$S \rightarrow A$
0x0D	DisputeOpen	$A \rightarrow \text{Auditor}$
0x0E	DisputeEvidence	$* \rightarrow \text{Auditor}$
0x0F	DisputeVerdict	$\text{Auditor} \rightarrow *$
0x10	PriceNegotiate	$A \leftrightarrow S$
0x11	PaymentChannel	$A \leftrightarrow S$
0x12	SettlementClose	$A/S \rightarrow *$
0x13	DiscoverAd	$A \rightarrow *$
0x14	DiscoverQuery	$A \rightarrow \text{Registry}$

The agent presents a token with three authority chains. The server intersects: the agent may book economy flights, up to $\min(\$2000, \$12340) = \$2000$ per trip, max 3/month, charged against both Alice’s personal limit and the corporate budget.

3) *SPIFFE/SPIRE Integration*: For workload identity in cloud-native environments, QASP authority chains can include SPIFFE Verifiable Identity Documents (SVIDs) [22] as an authority chain entry. The SVID provides platform attestation (“this agent is running in namespace X on cluster Y”), while QASP provides the capability authorization. This follows the SPIFFE/SPIRE pattern of separating workload identity from application-layer authorization.

I. Request-Grant-Revoke Flow

Table IV lists the QASP message types that govern the resource lifecycle.

J. Cryptographically Composable Delegation Chains

Delegation allows an agent to grant a subset of its capabilities to another agent, which is critical for multi-agent workflows. QASP formalizes delegation as a *token algebra* with provable security properties.

1) *Token Algebra*: Let T denote the space of valid capability tokens. We define three operations:

- **Intersection** ($T_a \cap T_b$): produces a token whose verb set is $V_a \cap V_b$ and whose constraints are the component-wise minimum (tightest) of both parents. Used when two authorization paths must both agree.
- **Attenuation** ($T' = \text{att}(T, \Delta)$): produces T' such that $V_{T'} \subseteq V_T$ and every constraint in T' is at most as permissive as in T . Δ specifies the narrowing. Escalation is algebraically impossible: $\forall d, \text{att}(T, d) \preceq T$.
- **Splitting** ($T \rightarrow T_1, T_2$): partitions a quantitative constraint (e.g., 2 vCPU-h $\rightarrow$ 1 + 1) such that $q_{T_1} + q_{T_2} \leq$

q_T . This enables parallel sub-delegation without over-commitment.

These operations ensure *least privilege by construction*: every derived token is provably no more powerful than its ancestor.

2) *Selective Disclosure*: In privacy-sensitive contexts a delegate may need to prove “I am authorized to `exec` on this resource” without revealing the full token (which may disclose spending limits, the delegator’s identity, or other constraints). QASP supports *selective disclosure* via:

- 1) **Hash-based commitment**: the full token is Merkle-ized; the delegate reveals only the branches relevant to the verifier’s query.
- 2) **Zero-knowledge proof of inclusion**: the delegate proves that the presented verb/resource pair is contained within a validly signed token without revealing remaining fields.

This balances authorization verification with data minimization, aligning with privacy regulations such as GDPR.

3) *Signature Aggregation for Deep Chains*: A delegation chain of depth d naively requires d separate ML-DSA-65 signature verifications. To reduce overhead, QASP supports *aggregate signatures*: the chain signatures are combined into a single compact proof that the entire chain is valid. For ML-DSA (a lattice-based scheme), we employ a *sequential aggregation* construction where each delegator signs the concatenation of the previous aggregate and the new token, yielding a single final signature that implicitly covers every link. Verification cost is $O(1)$ signature checks plus $O(d)$ hash computations, dramatically reducing overhead for deep chains.

4) *Delegation Rules*: The delegation rules enforce:

- 1) **Attenuation only**: A delegatee’s permissions must be a strict subset of the delegator’s—escalation is impossible by the token algebra.
- 2) **Depth limit**: `maxDelegationDepth` in the original token limits chain length.
- 3) **Chain verification**: Aggregate signature covers the full chain; individual verification is possible as fallback.
- 4) **Revocation cascades**: Revoking any token revokes all descendants.

For example, Owner Alice issues token T_0 to Agent Alpha (2 vCPU-hours, \$2.50, depth 2). Alpha delegates $T_1 = \text{att}(T_0, \Delta_1)$ to Beta (1 vCPU-hour, \$1.00, depth 1). Beta delegates $T_2 = \text{att}(T_1, \Delta_2)$ to Gamma (0.5 vCPU-hours, \$0.50, non-delegatable). The server verifies the aggregate signature and confirms monotonic attenuation at each level.

K. Accountability Layer: Metering, Receipts, and Disputes

Resource usage is tracked via periodic `MeterReport` messages from the server containing cumulative units consumed, cost accrued, and an ML-DSA-65 signature. Agents respond with signed `MeterAck` messages. If constraints are exceeded, the server issues a `ResourceSuspend`.

1) **Deterministic Usage Receipts:** Each metering exchange produces a *usage receipt* in canonical CBOR form:

```
Receipt = {
  token_id:  bytes,      ; UUID of capability token
  seq:       uint,       ; monotonic sequence number
  window:    [t_start, t_end],
  units:     decimal,    ; cumulative consumption
  cost:      decimal,    ; cumulative spend
  prev_hash: bytes,      ; SHA-384 of previous receipt
  server_sig: bytes,     ; ML-DSA-65 signature
  agent_sig: bytes      ; ML-DSA-65 counter-signature
}
```

The `prev_hash` field creates a *hash chain*: tampering with any receipt invalidates all subsequent hashes, providing tamper-evidence without requiring a blockchain. Both parties retain the chain; either can present it to a third-party auditor.

2) **Dispute Protocol:** When agent and server disagree on usage (e.g., the agent claims 1.2 vCPU-h but the server billed 1.8), QASP defines a structured escalation:

- 1) **DisputeOpen** (0x0D): The disputing party submits the contested receipt range and its local receipt chain to an *Auditor* (a mutually agreed third party or DAO).
- 2) **DisputeEvidence** (0x0E): Both parties submit their full receipt chains and any *replayable traces* (signed logs of individual API calls within the disputed window).
- 3) **DisputeVerdict** (0x0F): The Auditor replays the traces against the token constraints, compares both receipt chains, and issues a binding verdict with cryptographic proof of the decision basis.

Evidence is *bounded*: each party submits at most k receipts and n trace entries (configurable per-token), preventing denial-of-service via evidence flooding.

3) **Non-Repudiation with Privacy:** Dual-signed receipts provide non-repudiation, but tool arguments may contain private data (e.g., medical queries, proprietary prompts). QASP addresses this tension:

- **Argument hashing:** Trace entries include $H(\text{args})$ rather than raw arguments; the full arguments are revealed *only* during dispute resolution and only to the Auditor under NDA/smart-contract terms.
- **Auditor-only decryption:** Sensitive argument fields are encrypted to the Auditor’s public key at trace time; neither the server nor the agent can read each other’s private arguments post-hoc, but the Auditor can reconstruct the full trace if needed.

L. Economic Incentive Design and Payment Settlement

Metering alone is insufficient—agents need a protocol for agreeing on prices and settling payments without requiring trust. QASP-Settle provides this economic layer.

1) **Pricing Negotiation Protocol:** Before resource consumption begins, the agent and server negotiate pricing via a `PriceNegotiate` (0x10) exchange:

- 1) **PriceRequest:** Agent requests a quote for a specific ARM resource/verb/constraint tuple.
- 2) **PriceOffer:** Server responds with a price schedule (per-unit rate, minimum commitment, volume discounts, currency).

- 3) **PriceCounterOffer** (optional): Agent proposes alternative terms.
- 4) **PriceAccept:** Both parties sign the agreed price schedule, which is embedded in the capability token’s pricing field.

The signed price schedule is binding: neither party can unilaterally change pricing during the token’s validity window. Price disputes are handled by the same Auditor mechanism as metering disputes, with the signed schedule as evidence.

2) **Payment Channels for Agent Micropayments:** Inspired by the Lightning Network [21], QASP-Settle supports *bidirectional payment channels* for high-frequency, low-value agent transactions:

- 1) **Channel Open** (0x11): Agent and server each deposit a commitment (signed balance allocation) establishing an initial channel state $S_0 = \langle \text{agent_balance}, \text{server_balance} \rangle$.
- 2) **Off-chain updates:** As the agent consumes resources, both parties co-sign incremental state updates $S_i = \langle \text{agent_balance} - \sum \text{cost}_i, \text{server_balance} + \sum \text{cost}_i \rangle$. Each update carries a monotonically increasing sequence number.
- 3) **Channel Close** (0x12): Either party submits the latest co-signed state S_n for settlement. A challenge period (default: 1 hour) allows the counterparty to submit a higher-sequence state if the closer attempted to use a stale state.
- 4) **Unilateral close:** If one party is unresponsive, the other submits its latest co-signed state after the timeout. The challenge period protects against fraud.

All state updates are ML-DSA-65 signed by both parties and hash-chained (via `prev_hash`) to the receipt chain, unifying metering and payment into a single tamper-evident log. This enables *thousands of micropayments per second* without on-chain overhead.

3) **Arbitration Protocol for Disputed Metering:** When the agent’s local metering and the server’s metering diverge, a full dispute resolution flow activates:

- 1) **Divergence detection:** Either party detects that the counterparty’s signed `MeterReport`/`MeterAck` disagrees with its local state by more than a configurable tolerance threshold ϵ .
- 2) **Grace period:** A 60-second reconciliation window where both parties exchange their full receipt chains and attempt automatic resolution. If chains agree except for rounding, the higher-sequence receipt wins.
- 3) **Formal dispute:** If reconciliation fails, the disputing party opens a `DisputeOpen` with:
 - Both receipt chains (agent’s and server’s view).
 - The divergence point (first receipt where chains disagree).
 - Replayable trace logs for the disputed window.
- 4) **Auditor analysis:** The Auditor independently replays the trace against the agreed price schedule and token constraints, producing a *computed usage* figure.

- 5) **Binding verdict:** The Auditor issues a `DisputeVerdict` containing:
- The authoritative usage and cost.
 - A payment adjustment directive (refund or additional charge).
 - Optionally, a *fault attribution* (if one party's metering is systematically inaccurate, they may be penalized).
- 6) **Enforcement:** The payment channel state is updated to reflect the verdict. If either party refuses, the Auditor's verdict serves as evidence for out-of-band enforcement (e.g., reputation systems, legal recourse).

M. Revocation at Scale

Revoking a credential in a system with deep delegation chains and many active sessions requires careful engineering. This section specifies QASP's revocation architecture.

1) *Push vs. Pull Revocation:* QASP supports both strategies:

- **Push (proactive):** The revoker broadcasts `TokenRevocation` (0x05) messages to all known holders and servers. This provides fast propagation but requires maintaining a distribution list. QASP-Discover's advertisement infrastructure doubles as a revocation broadcast channel.
- **Pull (on-demand):** Servers check revocation status before accepting a token. QASP defines two pull mechanisms:
 - **QASP-CRL:** A periodically published Certificate Revocation List, ML-DSA-65 signed by the issuer, distributed as a CBOR array of revoked token IDs with revocation timestamps. Servers cache CRLs with a configurable refresh interval (default: 5 minutes).
 - **QASP-OCSP:** An online status check where the server sends a token ID to the issuer's OCSP responder and receives a signed "good"/"revoked"/"unknown" response. The response includes a `nextUpdate` timestamp for stapling: servers cache the response and present it to agents as proof of status, avoiding repeated round-trips.

Both CRL and OCSP responses use ML-DSA-65 signatures, maintaining PQ security throughout the revocation infrastructure.

2) *Grace Periods for In-Flight Transactions:* Immediate revocation can disrupt legitimate in-flight work. QASP defines three revocation urgency levels:

- 1) **Immediate** (`urgency=critical`): Token is invalid upon receipt of revocation. In-flight transactions are terminated. Used for key compromise.
- 2) **Graceful** (`urgency=normal`, default): A grace period (default: 5 minutes) allows in-flight transactions to complete. New transactions are rejected immediately; ongoing ones may finish within the grace window.

Servers MUST NOT extend the grace period beyond the token's original expiry.

- 3) **Scheduled** (`urgency=planned`): Revocation takes effect at a specified future time. Used for planned credential rotation.

3) *Revocation Cascade Implementation:* Consider the scenario: Owner revokes Agent Alpha's certificate. Alpha has 50 active delegation chains across 20 servers. The cascade proceeds:

- 1) **Owner signs revocation:**
`Revoke(Alpha's cert/DID, urgency, reason)` signed by owner's ML-DSA-65 key.
- 2) **Broadcast phase:** The revocation is pushed to:
 - All servers Alpha has active sessions with (via the session registry maintained during QASP-Shake).
 - The QASP-CRL distribution point.
 - The DID resolver network (if using `did:qasp`).
- 3) **Server-side cascade:** Each server, upon receiving Alpha's revocation:
 - Identifies all tokens where Alpha appears in the delegation chain (via the `issuer` and `parent_ref` fields in its token store).
 - Marks those tokens and ALL their descendants as revoked.
 - Sends `RevocationNotice` (0x06) to all agents holding affected tokens.
- 4) **Convergence:** Servers that missed the push revocation will discover it at their next CRL refresh or OCSP check. The maximum revocation propagation delay is bounded by `max(CRL_refresh_interval, push_delivery_timeout)`.
- 5) **Audit trail:** Each revocation event (original and cascaded) is logged with timestamp, affected token IDs, and the revocation chain, enabling post-hoc verification that the cascade completed correctly.

N. Message Format

All QASP messages use a common binary frame: 2-byte magic (0x5141, "QA"), 1-byte version, 1-byte message type, 4-byte big-endian payload length, variable-length CBOR payload, and a 48-byte HMAC-SHA-384 integrity tag computed over the header and payload using the session key.

O. Formal Specification Elements

1) *CDDL Schemas for CBOR Messages:* All QASP messages are formally specified using CDDL (Concise Data Definition Language, RFC 8610 [20]). The core message envelope:

```
qasp-message = {
  magic:      0x5141,
  version:    uint .size 1,
  msg_type:   uint .size 1,
  payload:    bytes,
  hmac:       bytes .size 48
}

capability-token = {
  version:     uint,
  token_id:    bytes .size 16,
  issued_at:   uint,
  expires_at:  uint,
```


TABLE V
QASP ERROR CODES

Code	Name	Recovery
0x01	VERSION_MISMATCH	Retry with offered version
0x02	SUITE_MISMATCH	Retry with common suite
0x03	AUTH_FAILED	Re-authenticate
0x04	TOKEN_EXPIRED	Request new token
0x05	TOKEN_REVOKED	Request new delegation
0x06	DEPTH_EXCEEDED	Flatten chain
0x07	CONSTRAINT_VIOLATED	Attenuate request
0x08	BUDGET_EXHAUSTED	Top up or close
0x09	RATE_LIMITED	Backoff and retry
0x0A	UPGRADE_REQUIRED	Use PQ suite
0x0B	DISPUTE_PENDING	Await verdict
0x0C	CHANNEL_CLOSED	Open new channel
0xFF	INTERNAL_ERROR	Retry or abort

```

    issuer:    tstr / bytes,    ; DID or fingerprint
    subject:   tstr / bytes,
    audience:  tstr / bytes,
    resource:  tstr,            ; ARM URI
    verbs:     [+ tstr],
    constraints: constraint-set,
    parent_ref: bytes / null,
    chain:     [* delegation-link],
    purpose:   tstr / null,
    toolchain: [* tstr] / null,
    authority_chains: [* authority-entry] / null,
    temporal_attenuation: attenuation-schedule / null,
    pricing:   price-schedule / null,
    signature: bytes
  }

  constraint-set = {
    ? time:      [uint, uint],
    ? quantity:  decimal,
    ? rate:      [decimal, uint],
    ? spend:     decimal,
    ? data_scope: tstr,
    ? purpose:   tstr
  }

  delegation-link = {
    delegator: tstr / bytes,
    token_ref: bytes .size 16,
    attenuation: constraint-set,
    signature: bytes
  }

  authority-entry = {
    role: tstr,
    chain: [+ delegation-link]
  }

  price-schedule = {
    currency: tstr,
    per_unit: decimal,
    ? min_commit: decimal,
    ? discounts: [* volume-discount]
  }

```

2) *Error Codes and Recovery*: QASP defines a structured error space:

Each error response includes a `retry_after` field (seconds) and a `detail` string for debugging. Clients **MUST** implement exponential backoff for retryable errors with a maximum of 5 retries.

IV. INTEROPERABILITY WITH A2A AND MCP

QASP is designed as a *security and authorization layer* that sits beneath existing agent orchestration protocols. It does not

replace A2A or MCP; it secures them.

A. Architectural Position

The interoperability stack is:

- 1) **Application layer**: A2A task orchestration / MCP tool invocation (unchanged).
- 2) **Authorization layer**: QASP-Cap tokens authorize each A2A task or MCP tool call.
- 3) **Security layer**: QASP-Shake provides the authenticated, PQ-encrypted channel.
- 4) **Transport layer**: TCP/QUIC (unchanged).

Existing A2A and MCP implementations require only a thin adapter to operate over QASP-secured channels, gaining PQ authentication, fine-grained authorization, and metered accountability without changing their application-layer semantics.

B. MCP-over-QASP: Tool Call Authorization

Each MCP tool call is wrapped in a QASP capability token as follows:

- 1) **Mapping**: The MCP tool name maps to an ARM resource URI: `mcp://{server}/tool/{tool_name}`. The MCP operation (`tools/call`) maps to ARM verb `exec`.
- 2) **Token wrapping**: Before sending an MCP `tools/call` request, the client constructs a QASP capability token scoped to the target tool, with constraints derived from the agent's authorization (spending limits, rate limits, purpose binding).
- 3) **Transport**: The MCP JSON-RPC message is sent as the payload of a QASP data frame over the QASP-Shake-secured channel. The QASP token ID is included in the MCP request's `_meta` field:

```

{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "web_search",
    "arguments": { "query": "..."},
    "_meta": {
      "qasp_token_id": "019...",
      "qasp_token": "<base64 CBOR token>"
    }
  }
}

```

The MCP server validates the QASP token before executing the tool, checking resource match, verb authorization, and constraint compliance. This adds authorization to MCP without modifying the MCP specification—the `_meta` field is explicitly reserved for extensions.

C. A2A-over-QASP Transport Profile

A2A task messages are transported over QASP-secured channels via a *transport profile*:

- 1) **Agent Card extension**: A2A Agent Cards include a `qasp` section with the agent's DID, QASP endpoint, and supported ciphersuites. Agents discovering each other

TABLE VI
MCP-TO-QASP AUTHORIZATION MAPPING

MCP Concept	QASP Equivalent
OAuth 2.0 token	QASP capability token
OAuth scope	ARM resource URI + verb set
Token refresh	Token re-issuance (attenuated)
Client credentials	Agent DID + ML-DSA-65 signature
Authorization server	Owner’s delegation authority
Token introspection	QASP-OCSP status check

TABLE VII
CRYPTOGRAPHIC ALGORITHM SELECTION

Purpose	Algorithm	Level
Signatures	ML-DSA-65 (FIPS 204)	Level 3
Key encapsulation	ML-KEM-768 (FIPS 203)	Level 3
Hybrid signatures	ML-DSA-65 + Ed25519	Level 3+
Hybrid KEM	ML-KEM-768 + X25519	Level 3+
Hashing	SHA-384 / SHA3-384	192-bit
Symmetric	AES-256-GCM	256-bit

via A2A Agent Cards can upgrade to QASP-secured communication.

- 2) **Task authorization:** Each A2A tasks/send or tasks/sendSubscribe carries a QASP token authorizing the task. The token’s ARM resource URI is `a2a://{agent}/task/{skill}`, and the verb is `exec`.
- 3) **Streaming:** A2A’s SSE-based streaming maps to QASP’s data frame stream. Each streamed artifact update is metered against the token’s constraints.
- 4) **Delegation:** When an A2A agent delegates a sub-task to another agent, it attenuates its QASP token for the delegate, providing cryptographic delegation tracking that A2A currently lacks.

D. MCP Authorization Mapped to QASP Capabilities

The mapping from MCP’s authorization model to QASP capabilities is:

This mapping is *additive*: systems running MCP with OAuth today can migrate to QASP tokens incrementally, gaining PQ security and delegation chains while maintaining backward compatibility via the hybrid profile.

V. SECURITY ANALYSIS

A. Algorithm Selection

Table VII summarizes the cryptographic algorithms used in QASP.

NIST Security Level 3 (~AES-192 equivalent) provides the optimal security/efficiency tradeoff for agent networking. Level 5 signatures (ML-DSA-87) are 40% larger (4627B vs. 3309B), which significantly impacts protocols where tokens are exchanged frequently. Level 3 is deemed sufficient against all known quantum attacks through at least 2060+ [6].

TABLE VIII
PQ VS. CLASSICAL PERFORMANCE OVERHEAD

Operation	Classical	PQ-only	Hybrid
Certificate size	~300B	~5.6KB	~5.9KB
Token size	~200B	~3.8KB	~4.1KB
Handshake bandwidth	~1KB	~12KB	~13KB
Sign time	~50 μ s	~1.5ms	~1.6ms
Verify time	~120 μ s	~0.8ms	~0.9ms
KEM encaps time	~100 μ s	~0.6ms	~0.7ms

TABLE IX
UNIFIED QASP VS. COMPOSED STACK

Metric	Composed	QASP
Handshake RTTs	3–4	1.5
Handshake bytes	~18KB	~13KB
Auth tokens per request	2–3	1
Signature verifications	4–6	2
Independent specs to impl.	4+	1
Delegation chain support	Bolted-on	Native
Metering integration	Separate	Unified
Consistent PQ coverage	Partial	Full

B. Performance Overhead

Table VIII quantifies the size and performance impact of post-quantum cryptography compared to classical approaches.

Mitigation strategies include CBOR encoding, token ID caching for repeat requests, certificate compression, and batched delegation chain verification.

C. Benchmarking Against TLS 1.3 + PQ Composed Approach

An alternative to QASP is composing existing protocols: TLS 1.3 with PQ ciphersuites (for channel security) + OAuth 2.0 (for authorization) + Macaroons (for delegation) + custom metering. We argue that QASP’s unified approach is superior:

Quantitative advantages: (1) *Fewer round-trips:* The composed approach requires a TLS handshake (1–2 RTT), then an OAuth token exchange (1 RTT), then a macaroon presentation—totaling 3–4 RTTs. QASP combines authentication, authorization, and channel establishment in 1.5 RTTs. (2) *Smaller bandwidth:* Carrying separate TLS certificates, OAuth tokens, and macaroons duplicates identity material. QASP’s unified token embeds authorization in the handshake. (3) *Consistent PQ coverage:* In the composed approach, TLS may use PQ ciphersuites, but OAuth tokens and macaroons remain classically signed—creating a PQ gap. QASP uses ML-DSA-65 for ALL signatures. (4) *Reduced attack surface:* Each independent protocol introduces its own vulnerability surface; composition creates seam vulnerabilities at protocol boundaries.

D. Threat Model and Mitigations

We consider five threat actors: a *Quantum Adversary* with CRQC access, a *Network Attacker* with MITM capability, a *Malicious Agent* with compromised software, a *Rogue Owner*

with legitimate but malicious credentials, and a *Compromised Server* controlling resource infrastructure.

HNDL attacks are mitigated by ML-KEM-768 providing quantum-resistant forward secrecy; past session keys cannot be recovered even if long-term keys are compromised.

Agent impersonation is prevented by requiring certificates or DID proofs signed by the owner’s ML-DSA-65 key; without the private key, valid credentials cannot be forged.

Token theft and replay is addressed by binding tokens to specific agent public keys (requiring proof of possession), enforcing expiry timestamps, and maintaining server-side token-use logs.

MITM attacks are defeated by mutual authentication in QASP-Shake with transcript binding—any modification is detected.

Privilege escalation via delegation is prevented by the token algebra (Section III-J), which makes escalation algebraically impossible, reinforced by server-side verification of the entire delegation chain.

Metering fraud is mitigated by hash-chained dual-signed receipts (Section III-K) with a structured dispute protocol providing replayable evidence to an independent Auditor.

E. Tool-Chaining Security

In multi-agent workflows, an agent often invokes a chain of tools: Agent A calls Tool 1, whose output feeds Tool 2 via Agent B. Without explicit safeguards, a capability granted for Tool 1 could be implicitly exercised against Tool 2—a form of *confused deputy* attack across tool boundaries.

1) *Capability Firebreaks*: QASP enforces that capabilities are *non-transitive across tool boundaries* by default. When Agent A’s tool invocation triggers a sub-invocation on a different service, the sub-invocation requires a *fresh token* explicitly re-minted (via attenuation) from Agent A’s original token. The re-minting process:

- 1) Agent A attenuates its token T to $T' = \text{att}(T, \Delta_{\text{tool2}})$, restricting resource URI, verb set, and constraints to exactly what Tool 2 needs.
- 2) T' is signed by Agent A and includes a `parent_ref` pointing to T .
- 3) The server hosting Tool 2 verifies the chain $T \rightarrow T'$ and confirms that T' ’s scope is appropriate for the requested action.

This “firebreak” ensures that a compromised intermediate tool cannot silently escalate access.

2) *Context Binding*: Every capability token includes two additional ARM fields:

- `purpose`: a structured label (e.g., `''data-analysis''`, `''model-training''`) that binds the token to a declared intent. Servers MAY reject tokens whose purpose does not match the service category.
- `allowed_toolchain`: an ordered list of tool/service classes (e.g., `[''compute'', ''storage'']`) that the token may traverse. If a tool invocation would exit the allowed toolchain, the server rejects the request.

TABLE X
EXAMPLE QASP CIPHERSUITE PROFILES

Profile	KEM	Signature
PQ-Strict	ML-KEM-768	ML-DSA-65
Hybrid-Transition	ML-KEM-768+X25519	ML-DSA-65+Ed25519
Classical-Compat	X25519	Ed25519

Together, firebreaks and context binding confine the blast radius of any single compromised agent or tool within a multi-step workflow.

F. Crypto Agility and Post-Quantum Transition

Cryptographic algorithms have finite lifespans. QASP is designed for *crypto agility*—the ability to adopt new algorithms without protocol-breaking changes.

1) *Ciphersuite Registry*: QASP maintains a versioned ciphersuite registry, analogous to TLS cipher suites. Each entry specifies a KEM algorithm, a signature algorithm, a hash function, and an AEAD cipher, identified by a 16-bit suite ID. New suites are added via registry updates without changing the protocol framing or handshake structure.

2) *Mandatory Downgrade Resistance*: During QASP-Shake, both parties advertise their supported suites in preference order. The protocol enforces:

- The selected suite **MUST** be the highest-security suite supported by both parties (server preference breaks ties).
- If the server supports any PQ suite and the client advertises only classical suites, the server **MUST** reject the connection and signal `UPGRADE_REQUIRED`, unless operating in an explicitly configured Classical-Compat deployment.
- The selected suite ID is included in the transcript hash, so any attacker who downgrades the suite is detected by the transcript signature.

3) *Transition Profiles*: To accommodate heterogeneous deployments, QASP defines three named profiles (Table X):

- **PQ-Strict**: Only post-quantum algorithms. Mandatory for high-security and government deployments.
- **Hybrid-Transition**: Concurrent PQ and classical algorithms; security holds if *either* is unbroken. Recommended default during the 2025–2035 transition window.
- **Classical-Compat**: Classical-only fallback for legacy interoperability. Tokens issued under this profile carry a `pq_warning` flag and a shorter maximum validity period (24h vs. 30d).

G. Formal Security Properties and Proofs

QASP’s security claims go beyond assertion—we outline formal adversary games and verification approaches.

1) *Adversary Games*: We define security in the hybrid PQ/classical setting via standard cryptographic games:

IND-CCA2 for QASP-Shake. The key exchange is IND-CCA2 secure if no probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can distinguish the session key from random with

non-negligible advantage. In hybrid mode, QASP derives the session key as $SS = \text{HKDF}(\text{ML-KEM}_{ss} \| X25519_{ss}, \dots)$. The hybrid construction is IND-CCA2 secure if *either* ML-KEM-768 is IND-CCA2 under Module-LWE [10] *or* X25519 is CDH-secure. Formally:

$$\text{Adv}_{\text{QASP-Shake}}^{\text{IND-CCA2}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{B}_1}^{\text{MLWE}} + \text{Adv}_{\mathcal{B}_2}^{\text{CDH}} + \text{negl}(\lambda) \quad (4)$$

EUF-CMA for tokens and certificates. No PPT adversary can produce a valid capability token or certificate without the signer’s private key. Under the Module-SIS hardness assumption for ML-DSA-65 [11]:

$$\text{Adv}_{\text{QASP-Token}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{B}}^{\text{MSIS}} + \text{negl}(\lambda) \quad (5)$$

Delegation chain safety. We model the token algebra as a partial order $(\mathcal{T}, \preceq)$ and prove that for any sequence of attenuation and splitting operations, the resulting token T' satisfies $T' \preceq T_{\text{root}}$. The proof is by structural induction on the chain depth, using the monotonicity of constraint intersection.

2) *Tamarin/ProVerif Modeling Approach:* We specify the QASP-Shake protocol in the applied pi-calculus for verification with ProVerif [23] and in multiset rewriting rules for Tamarin [24]. The properties to be verified:

- **Secrecy:** The session key SS is not derivable by the adversary (even with quantum oracle access to hash functions, modeled as random oracles).
- **Authentication:** If an agent completes the handshake believing it spoke to server S , then S indeed participated (injective agreement).
- **Forward secrecy:** Compromise of long-term keys does not reveal past session keys.
- **Delegation safety:** No sequence of protocol messages produces a token T' with $T' \not\preceq T_{\text{root}}$ (no escalation).

The Tamarin model encodes ML-KEM as an abstract IND-CCA2-secure KEM and ML-DSA as an abstract EUF-CMA-secure signature scheme, enabling symbolic verification of protocol logic independent of the specific lattice constructions. Preliminary models (available in the companion repository) verify secrecy and authentication properties. Full formal verification of all properties is ongoing work targeted for Q3 2026.

H. Formal Security Properties Summary

QASP provides (subject to implementation correctness):

- 1) **IND-CCA2 key exchange**—ML-KEM-768 is IND-CCA2 secure under Module-LWE [10]; hybrid mode provides dual security.
- 2) **EUF-CMA signatures**—ML-DSA-65 is existentially unforgeable under Module-SIS [11].
- 3) **Forward secrecy**—Ephemeral KEM keys per session.
- 4) **Capability safety**—Token algebra ensures no delegation chain can escalate permissions (monotonic attenuation), proved by structural induction.
- 5) **Non-repudiation**—Hash-chained dual-signed receipts and signed capability tokens.

- 6) **Tool-chain confinement**—Capability firebreaks prevent transitive authority leakage across tool boundaries.
- 7) **Downgrade resistance**—Transcript-bound suite negotiation prevents forced algorithm downgrade.
- 8) **Economic fairness**—Payment channels with challenge periods prevent unilateral fraud; Auditor verdicts provide binding dispute resolution.

VI. STANDARDIZATION ROADMAP

QASP is designed from the outset to be standardizable. This section outlines the path from research prototype to interoperable specification.

A. RFC-Style Specification Companion

The QASP specification will be authored as an Internet-Draft using IETF conventions:

- **Normative language:** Requirements are expressed using RFC 2119 keywords (MUST, SHOULD, MAY) [17]. For example: “A server **MUST** reject a delegation token whose depth exceeds `maxDelegationDepth`.” “An implementation **SHOULD** support Hybrid-Transition as the default profile.” “An implementation **MAY** support Classical-Compat for legacy interoperability.”
- **CDDL schema tables:** Every message type and token structure is specified as a CDDL schema (Section III-O), ensuring unambiguous wire-format definitions. CDDL schemas enable automated codec generation and cross-implementation compatibility testing.
- **ABNF grammar:** Message framing and URI syntax are specified using ABNF (RFC 5234). The ARM resource URI grammar:

```
qasp-uri = "qasp://" provider "/" resource
          [ "/" subresource ]
provider = 1*( ALPHA / DIGIT / "-" / "." )
resource = 1*( ALPHA / DIGIT / "-" / "_" )
subresource = 1*( ALPHA / DIGIT / "-" / "_" / "/" )
```

B. Reference Stack and Conformance Suite

We plan to deliver:

- 1) **Reference implementation:** Open-source Python library (`libqasp`) implementing all six sub-protocols, using `oqs-python` for PQ primitives and `cbor2` for serialization.
- 2) **Conformance test suite:** A collection of test vectors and protocol traces covering:
 - Handshake success/failure for each ciphersuite profile (including version mismatch and timeout recovery)
 - Token creation, attenuation, splitting, aggregation, threshold delegation, and temporal evolution
 - Receipt chain validation, payment channel state transitions, and dispute replay
 - Revocation cascades across multiple servers
 - DID resolution and owner binding verification
 - MCP-over-QASP and A2A-over-QASP interoperability

- Edge cases: expired tokens, exceeded depth, downgrade attempts, cross-domain delegation

3) **Interoperability harness:** Docker-based test environment where two independent implementations can run the conformance suite against each other.

C. Testable State Machines

Both the QASP-Shake handshake (Section III-F1) and the token verification flow are specified as *deterministic finite state machines* (DFSMs) with explicitly enumerated states, transitions, and error conditions. These state machines:

- Enable *model checking* (e.g., via TLA+ or SPIN) to verify liveness and safety properties before implementation.
- Serve as the *canonical reference* for implementers—any ambiguity in prose is resolved by the state machine.
- Are directly translatable to executable code via code generation from the state-machine specification.

The QASP-Shake state machine has 8 states (Idle, HelloSent, HelloReceived, SHSent, Authenticating, Verifying, Established, Error) and 14 transitions (Table III). The token verification state machine has 5 states and 8 transitions. The payment channel state machine has 4 states (Unopened, Open, Closing, Closed) and 6 transitions. Full diagrams and transition tables will be included in the Internet-Draft.

D. Timeline

- 1) **Q1 2026:** Proof-of-concept implementation, this paper, and Tamarin/ProVerif models.
- 2) **Q2 2026:** CDDL schema finalization, DID method registration, and reference implementation release.
- 3) **Q3 2026:** Conformance suite, interoperability testing, and formal verification completion.
- 4) **Q4 2026:** Internet-Draft submission to IETF; A2A/MCP bridge adapters released.

VII. IMPLEMENTATION PLAN

A. Proof-of-Concept Scope

The demonstration shows two agents negotiating compute access using QASP with real post-quantum cryptographic operations via `liboqs` [12].

B. Technology Stack

The implementation uses Python 3.12+ with `oqs-python` for post-quantum cryptography, `cbor2` for serialization, raw TCP sockets for transport (demonstrating protocol framing), a custom simplified X.509-PQ certificate profile and `did:qasp` resolver, and a terminal CLI with protocol trace visualization.

C. Demo Scenarios

Scenario 1 (Direct Resource Request): Owner Alice creates an agent Alpha with ML-DSA-65 credentials and a `did:qasp` identity. Alpha discovers Server Beta via QASP-Discover, performs QASP-Shake, negotiates pricing, requests 1 vCPU for 2 hours, receives a capability token, uses the resource with periodic metering and micropayment settlement, and releases it with a final usage summary.

Scenario 2 (Delegation Chain): Agent Alpha, holding token T_0 from Scenario 1, creates sub-agent Gamma and delegates an attenuated token $T_1 = \text{att}(T_0, \Delta)$. Gamma connects to Server Beta, presents the full delegation chain (with aggregate signature), and uses compute resources metered against T_1 's constraints.

Scenario 3 (Tool-Chain with Firebreak): Agent Alpha invokes Tool 1 on Server Beta, then needs to chain into Tool 2 on Server Gamma. Alpha re-mints an attenuated, purpose-bound token T' for Tool 2, demonstrating capability firebreaks and context binding in a multi-hop workflow.

Scenario 4 (MCP-over-QASP): An MCP client uses QASP-Shake to connect to an MCP server, wrapping each `tools/call` in a QASP capability token, demonstrating the interoperability bridge.

D. Evaluation Metrics

Measurements include handshake latency (ClientHello to Finished), handshake bandwidth, token creation and verification times, PQ vs. classical overhead comparison, delegation chain depth impact on verification time (with and without signature aggregation), receipt chain validation throughput, payment channel throughput (state updates per second), revocation cascade propagation time, and comparison with the composed TLS+OAuth+Macaroons stack.

VIII. CONCLUSION

We have presented QASP, a Quantum-Armored Service Protocol that addresses critical gaps in existing agent communication protocols. By combining decentralized identity via `did:qasp` with optional CA anchoring, PQ-secured service discovery via QASP-Discover, a Universal Agent Resource Model (ARM) providing a canonical vocabulary for capabilities and constraints, fine-grained capability tokens with formal token algebra ensuring least privilege by construction—including aggregation, threshold delegation, temporal evolution, and cross-domain delegation, cryptographically composable delegation chains with selective disclosure and signature aggregation, multi-owner organizational delegation with hierarchical policy composition, a tamper-evident accountability layer with hash-chained receipts and structured dispute resolution, an economic settlement layer with pricing negotiation and Lightning-inspired micropayment channels, tool-chaining security via capability firebreaks and context binding, crypto agility with mandatory downgrade resistance and named transition profiles, explicit interoperability bridges to A2A and MCP (positioning QASP as security layer, not replacement), formal specification elements including CDDL schemas, state machines, error codes, and ABNF grammars, formal security arguments via adversary games with Tamarin/ProVerif modeling, and post-quantum cryptography using NIST-standardized ML-DSA-65 and ML-KEM-768, QASP provides a comprehensive foundation for secure autonomous agent communication and resource authorization that is resilient against both classical and quantum threats, while demonstrating quantitative advantages over composed protocol stacks.

The protocol’s hybrid PQ/classical mode enables graceful transition, while the capability-based authorization model with monotonic attenuation ensures the principle of least privilege in multi-agent delegation scenarios. Our standardization roadmap—including RFC-style normative specification, CDDL schema tables, testable state machines, and a conformance suite—charts a clear path from research prototype to interoperable standard. Our implementation plan demonstrates feasibility through a proof-of-concept using the Open Quantum Safe library.

REFERENCES

- [1] Gartner, “Top Strategic Technology Trends 2025: Agentic AI,” Gartner Research, Oct. 2024.
- [2] Google, “Agent-to-Agent Protocol (A2A),” 2025. [Online]. Available: <https://github.com/a2aproject/A2A>
- [3] Anthropic, “Model Context Protocol (MCP),” 2024. [Online]. Available: <https://modelcontextprotocol.io>
- [4] ANP Community, “Agent Network Protocol,” 2024. [Online]. Available: <https://github.com/agent-network-protocol/AgentNetworkProtocol>
- [5] Cisco AGNTCY, “Agent Gateway Protocol (AGP),” 2025. [Online]. Available: <https://agentgateway.dev>
- [6] NIST, “Transition to Post-Quantum Cryptography Standards,” NIST IR 8547, 2024.
- [7] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM),” FIPS 203, 2024.
- [8] NIST, “Module-Lattice-Based Digital Signature Standard (ML-DSA),” FIPS 204, 2024.
- [9] R. Barnes, K. Bhargavan, B. Lipp, and C. Wood, “Hybrid Public Key Encryption,” RFC 9180, IRTF, 2022.
- [10] R. Avanzi et al., “CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation,” NIST PQC Round 3, 2021.
- [11] L. Ducas et al., “CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation,” NIST PQC Round 3, 2021.
- [12] Open Quantum Safe Project, “liboqs,” [Online]. Available: <https://openquantumsafe.org>
- [13] D. J. Bernstein and T. Lange, “Post-Quantum Cryptography,” *Nature*, vol. 549, pp. 188–194, 2017.
- [14] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multi-programmed Computations,” *Commun. ACM*, vol. 9, no. 3, pp. 143–155, 1966.
- [15] M. S. Miller, “Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control,” Ph.D. dissertation, Johns Hopkins University, 2006.
- [16] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrabie, and M. Lentzner, “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” in *Proc. NDSS*, 2014.
- [17] S. Bradner, “Key words for use in RFCs to Indicate Requirement Levels,” RFC 2119, IETF, 1997.
- [18] W3C, “Decentralized Identifiers (DIDs) v1.0,” W3C Recommendation, 2022. [Online]. Available: <https://www.w3.org/TR/did-core/>
- [19] S. Cheshire and M. Krochmal, “DNS-Based Service Discovery,” RFC 6763, IETF, 2013.
- [20] H. Birkholz, C. Vigano, and C. Bormann, “Concise Data Definition Language (CDDL),” RFC 8610, IETF, 2019.
- [21] J. Poon and T. Dryja, “The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments,” 2016.
- [22] SPIFFE, “Secure Production Identity Framework for Everyone,” [Online]. Available: <https://spiffe.io>
- [23] B. Blanchet, “Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif,” *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, 2016.
- [24] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” in *Proc. CAV*, 2013.
- [25] R. Bendlin et al., “Threshold Signatures from Lattice-Based Assumptions,” in *Proc. CRYPTO*, 2024.